

Timeline Execution Plan – Lost and Found Management System

Week 1 – Requirement Analysis

- Finalize project problem statement
 - Identify stakeholders & use cases
 - List functional & non-functional requirements
-

Week 2 – System Design

- Draw **ER Diagram**
 - Create **Relational Schema** with constraints
 - Prepare **Data Flow Diagram (DFD)**
 - Design basic UI wireframes
-

Week 3 – Database Setup

- Install & configure **PostgreSQL**
 - Create database LostAndFoundDB
 - Write CREATE TABLE scripts for Users, LostItems, FoundItems, Matches, Admins
 - Test schema with sample data
-

Week 4 – Backend Development

- Set up backend environment (Node.js / PHP / Flask)
 - Build APIs for:
 - Insert lost/found item
 - Search & retrieve items
 - Admin verification
 - Connect backend with PostgreSQL
-

Week 5 – Frontend Development

- Create forms (Lost item, Found item, Search)
 - Build **Admin dashboard**
 - Add basic styling (HTML, CSS, JS)
-

Week 6 – Integration & Testing

- Connect frontend ↔ backend ↔ database
 - Perform **unit testing** (API, DB queries)
 - Perform **integration testing** (end-to-end workflow)
 - Optimize queries with indexes
-

Week 7 – Documentation

- Prepare **README.md** (features, setup, usage)
 - Include **ER Diagram & Schema**
 - Write **User manual** (how to use system)
 - Add screenshots of working system
-

Week 8 – Deployment & Finalization

- Deploy locally (XAMPP/Node server)
 - (Optional) Deploy on **Heroku / Render / AWS**
 - Push code to **GitHub repository**
 - Final project presentation & viva prep
-

✓ This makes your plan look structured and time-bound, which professors usually appreciate.

✂ 1. Database (Core of DBMS Project)

- **PostgreSQL** (main DBMS)
 - Learn:
 - CREATE TABLE, INSERT, SELECT, UPDATE, DELETE
 - Primary keys, foreign keys, constraints
 - Joins, subqueries, indexes
 - Simple triggers & stored procedures

(If PostgreSQL feels heavy, you can also use MySQL, but PostgreSQL is more advanced.)

📄 2. Backend (to connect database with UI)

Choose **one** (depends on your preference or college environment):

- **Node.js + Express (JavaScript)**
 - Popular, easy to connect with PostgreSQL using pg package.
- **PHP (with XAMPP/WAMP)**
 - Very common in college DBMS projects.
- **Python (Flask/Django)**
 - Easy to learn and use for APIs.

Backend responsibilities:

- Connect to PostgreSQL
 - Handle form submissions (lost/found item)
 - Perform queries (insert/search/match)
 - Admin authentication
-

🌐 3. Frontend (User Interface)

- **HTML** → structure of forms & pages
- **CSS** → styling for user-friendly design

- **JavaScript** → form validation, interactivity

(Optional): Use **Bootstrap** for quick responsive design.

4. Version Control

- **Git & GitHub**
 - To manage versions of your project and collaborate.
 - Learn basics: git init, git add, git commit, git push, git pull.
-

5. Deployment (Optional but impressive)

- **Local Deployment** → XAMPP / Node server
 - **Cloud Deployment** → Heroku / Render / GitHub Pages (for frontend only)
-

6. Tools & Utilities

- **DB Diagram Tool** → draw ER Diagrams (dbdiagram.io, Lucidchart, Draw.io)
 - **Postman** → test your backend APIs (if using Node/Flask)
 - **VS Code** → coding environment
-

Summary (Minimal Learning Path):

- PostgreSQL (database)
- HTML, CSS, JavaScript (frontend)
- One backend (Node.js OR PHP OR Flask)
- Git & GitHub